

Resumen - Paradigmas de Programación

1. GENERAL

Paradigma vs Lenguaje de Programación

Lenguaje	Paradigma
Implementación concreta (Haskell, Prolog, Java)	Marco conceptual / forma de pensar
Sintaxis, herramientas, compilador	Reglas y convenciones de modelado

Dos diferencias clave:

1. Un paradigma es una forma de pensar el problema; un lenguaje es una herramienta que lo implementa.
2. Un mismo lenguaje puede implementar varios paradigmas (lenguajes híbridos).

Lenguajes híbridos: Scala (funcional + OO), Python (imperativo + funcional + OO), Kotlin, C++.

Paradigmas vistos en clase:

- **Funcional** → Haskell
- **Lógico** → Prolog
- **Orientado a Objetos** → Java

Imperativo vs Declarativo

- **Imperativo:** paso a paso, el programador indica *cómo* hacerlo. Hay secuencias, estructuras de control, variables y estado global.
- **Declarativo:** se especifica *qué* se quiere, se delega el *cómo*. Los paradigmas funcional y lógico son declarativos.

2. PARADIGMA FUNCIONAL - HASKELL

Principios fundamentales

Unicidad y Existencia (como funciones matemáticas):

- **Existencia:** para todo valor de entrada existe una salida.
- **Unicidad:** para una entrada hay una y solo una salida posible.
- El paradigma lógico puede tener múltiples respuestas para una misma entrada.

Transparencia Referencial → Principio de Sustitución:

- La salida *solo* depende de la entrada (no hay estado global ni efecto).
- Las funciones son **independientes, determinísticas y sin efecto**.
- Dado que $f(x)$ siempre devuelve lo mismo, se puede reemplazar $f(x)$ por su resultado directamente.

Tipos de datos en Haskell

```
Number -- Int, Float, Double   Ej: 4, 4.5
String -- [Char]               Ej: "Hola"
Bool   -- True, False
Char   -- 'a', 'b'
```

Inferencia de tipos

Haskell deduce el tipo de una función a partir de cómo se usan sus parámetros:

```
ff x y = x && not y
-- y entra a not → Bool; x entra a && → Bool
-- ff :: Bool -> Bool -> Bool
```

`:t funcion` muestra el tipo. `:t funcion argumento` muestra el tipo de la aplicación parcial.

Variables de tipo: cuando el tipo es genérico se usa una letra minúscula.

```
id :: a -> a
ignorarPrimero :: a -> b -> b
```

Type Classes (restricciones de tipo)

```
dobles :: Num a => a -> a
(==)   :: Eq a => a -> a -> Bool
(<)    :: Ord a => a -> a -> Bool
(/)    :: Fractional a => a -> a -> a
even   :: Integral a => a -> Bool
```

Las funciones no tienen clase asociada; no se pueden sumar ni comparar entre sí.

Pattern Matching

Evalúa de arriba hacia abajo; primero constantes, luego variables. El orden importa.

```
color "banana" = "amarillo"
color "manzana" = "rojo"
color _        = "gris"      -- caso default con variable anónima

factorial 0 = 1
factorial n = factorial (n-1) * n
```

Si ningún patrón matchea y no hay caso default, se lanza una excepción (función parcial → viola la existencia).

Función Partida (Guardas)

```
f :: Int -> Int
f x
  | x < -1          = -1
  | -1 <= x && x <= 1 = 1
  | otherwise      = -1
```

Guarda vs Pattern Matching: las guardas usan condiciones booleanas; el pattern matching usa la estructura o valor del dato.

Función Parcial

Está definida solo en parte del dominio (viola la existencia). Ejemplo: `diaDeSemana 8` lanza excepción porque solo están definidos del 1 al 7.

Precedencia de operadores

Las funciones **prefijas** tienen más precedencia que los operadores o funciones **infijas**.

```
even length "alfonsina"    -- ERROR: se interpreta como (even length) "alfonsina"
even (length "alfonsina")  -- OK: primero length, luego even

8 + 7 * 2    -- = 22 (el * tiene más precedencia que el +)
```

Usar paréntesis para forzar el orden de evaluación deseado.

Alias y Tipos propios

Type (alias): renombra un tipo existente para dar semántica.

```
type Nombre = String
type Edad   = Int
```

Tupla: agrupa valores de tipos distintos. Anónima, pierde semántica.

```
type Persona = (Nombre, Edad)
edad :: Persona -> Edad
edad (_, e) = e    -- pattern matching sobre tupla
fst (x, _) = x
snd (_, y) = y
```

Data: define un tipo propio con nombre, constructor y funciones de acceso generadas automáticamente. Más expresivo.

```
data Estudiante = UnEstudiante {
  nombre :: String,
  legajo  :: String,
  nota    :: Int
} deriving (Show, Eq)

-- Uso
juanita = UnEstudiante "Juana" "L004100-3" 8
nota juanita    -- 8

-- Crear una copia con campo modificado
cambiarNota nuevaNota (UnEstudiante n l _) = UnEstudiante n l nuevaNota
```

Diferencia Tupla vs Data: el Data tiene nombre propio, constructor con semántica y genera funciones de acceso; la tupla es anónima y menos expresiva. Para modelar entidades del dominio, preferir Data.

Listas

```
[1, 2, 3]
1 : [2, 3]      -- cons
1 : 2 : 3 : []

head, tail, last, init, length, reverse
take n lista, drop n lista
elem x lista
++ , sort, sum, maximum, minimum
```

Pattern matching sobre listas:

```
length []      = 0
length (_:xs) = 1 + length xs
```

Aplicación Parcial vs Composición

Composición (.): encadena funciones — la salida de una es la entrada de la siguiente.

```
(.) :: (b->c) -> (a->b) -> (a->c)

promociona = not . esMenorAOcho . nota
-- equivale a: promociona alumno = not (esMenorAOcho (nota alumno))
```

Aplicación Parcial: llamar una función con *menos* argumentos de los que pide, generando una nueva función.

```
esMayor    = (>= 18)
doble      = (2 *)
alMenosCero = (max 0)

-- Con funciones infijas se puede aplicar primer o segundo parámetro
(== 'a') :: Char -> Bool
(+ 5)    :: Num a => a -> a
```

Diferencia clave: la composición encadena funciones; la aplicación parcial fija uno o más parámetros generando otra función.

Currificación: toda función en Haskell recibe en realidad un parámetro a la vez. $f\ x\ y\ z$ es equivalente a $((f\ x)\ y)\ z$. Esto es lo que hace posible la aplicación parcial.

Point Free: definición de función sin mencionar los parámetros, usando composición y aplicación parcial.

```
edades          = map edad
adultosMayores = filter esMayor
```

Recursividad

Una abstracción definida sobre sí misma. Dos tipos:

- Sobre **estructura de datos** (listas): caso base lista vacía.
- Sobre **valores numéricos**: caso base 0 o 1.

```
factorial 0 = 1
factorial n = n * factorial (n-1)

length []      = 0
length (_:xs) = 1 + length xs

take 0 _       = []
take n []      = []
take n (x:xs) = x : take (n-1) xs
```

Orden Superior

Funciones que **reciben o devuelven otras funciones**. Separan el algoritmo de la implementación particular. Permiten mayor declaratividad, cohesión y expresividad.

```
map :: (a->b) -> [a] -> [b]
-- aplica f a cada elemento
```

```
map doble [1,2,3] -- [2,4,6]

filter :: (a->Bool) -> [a] -> [a]
-- conserva los que cumplen la condición
filter esMayor personas

all :: (a->Bool) -> [a] -> Bool
-- True si todos cumplen (lazy: corta al primer False)

any :: (a->Bool) -> [a] -> Bool
-- True si alguno cumple (lazy: corta al primer True)

foldr :: (b->a->a) -> a -> [b] -> a
-- reduce una lista con semilla, asociativo a derecha
foldr (+) 0 [1,2,3] -- 6

foldl :: (a->b->a) -> a -> [b] -> a
-- reduce asociativo a izquierda
```

Diferencia foldr vs foldl — asociatividad:

```
-- foldr: asocia a la derecha (1 - (2 - 3))
foldr1 (-) [1,2,3] -- = 1 - (2 - 3) = 2

-- foldl: asocia a la izquierda ((1 - 2) - 3)
foldl1 (-) [1,2,3] -- = (1 - 2) - 3 = -4
```

foldl1 / foldr1: igual que foldl/foldr pero sin semilla — usan el primer elemento como valor inicial. Fallan con lista vacía.

```
foldl1 :: (a->a->a) -> [a] -> a
foldl1 f (x:xs) = foldl f x xs

foldr1 `max` [1,2,3] -- 3
```

concatMap: aplica una función que devuelve lista a cada elemento y concatena los resultados.

```
concatMap :: (a -> [b]) -> [a] -> [b]
concatMap autores todasLasObras -- lista plana de todos los autores
```

Otras funciones de Orden Superior útiles:

```
zipWith :: (a->a->a) -> [a] -> [a] -> [a]
zipWith (+) [1,2,3] [4,5,6] -- [5,7,9]
```

```
iterate :: (a->a) -> a -> [a]    -- genera lista infinita aplicando f repetidamente
iterate (*2) 1    -- [1,2,4,8,16,...]

flip :: (a->b->c) -> b -> a -> c    -- invierte el orden de los dos primeros
parámetros
flip (/) 2 10    -- 10/2 = 5.0

($) :: (a->b) -> a -> b    -- aplicación de función, menor precedencia que todo
($) div 10 2    -- equivale a div 10 2 = 5
not . even $ length "hola"    -- aplica (not.even) al resultado de length "hola"
```

Lambda (función anónima):

```
map (\x -> x + 1) [1,2,3]
filter (\p -> edad p > 18) personas
```

Ejemplo completo con Orden Superior:

```
alimentosPocoCaloricos = map alimento . filter pocoCalorico
pocoCalorico = (<= 100) . calorias
```

Evaluación Lazy vs Eager

- **Eager:** evalúa los argumentos *antes* de ejecutar la función.
- **Lazy** (Haskell): retrasa la evaluación hasta que es estrictamente necesaria. Permite listas infinitas.

```
-- Lazy: no evalúa el segundo elemento de la tupla
fst (2*40, div 600 0)    -- devuelve 80 sin error

-- Lazy: toma solo el primer elemento de una lista infinita
head (iterate (*2) 1)    -- devuelve 1
```

3. PARADIGMA LÓGICO - PROLOG

Conceptos fundamentales

Universo Cerrado: todo lo que no está en la base de conocimiento es **FALSO** (no existe "desconocido").

Taxonomía completa de individuos y predicados:

```
Individuo
├─ Simple
│   ├── Átomo      (empieza con minúscula: juan, sandman)
│   └─ Número      (42, 3.14)
```

```

└─ Compuesto
  └─ Lista      ([a, b, c])
  └─ Functor    (punto(3, 4), fecha(2024, junio, 8))

Predicado
└─ Propiedad    (aridad 1: esComic(sandman).)
└─ Relación     (aridad >1: escribio(gaiman, sandman).)

Cláusula
└─ Hecho (Axioma)  (verdad directa)
└─ Regla          (conclusión inferida: cabeza :- cuerpo.)

Consulta
└─ Individual     (todos instanciados → V/F)
└─ Existencial    (con variables libres → liga individuos)

```

Base de Conocimiento

```

escribio(elsaBornemann, socorro).
escribio(neilGaiman, sandman).
escribio(neilGaiman, buenosPresagios).
escribio(terryPratchett, buenosPresagios).

esComic(sandman).
esComic(watchmen).

```

Consultas

Individual (todos los argumentos instanciados → V/F):

```

?- escribio(neilGaiman, sandman).    % true
?- escribio(neilGaiman, it).         % false

```

Existencial (variables libres → Prolog liga individuos):

```

?- escribio(Artista, sandman).        % Artista = neilGaiman
?- escribio(neilGaiman, Obra).        % Obra = sandman ; Obra = buenosPresagios
?- escribio(_, buenosPresagios).     % true

```

Diferencia clave: la consulta individual verifica si una relación se cumple para individuos dados; la existencial busca qué individuos satisfacen la relación.

Las variables en Prolog van en **Mayúscula**. La variable anónima `_` ignora el valor.

Reglas


```
% P ^ Q => R se escribe R :- P, Q.
esArtistaDeNovenoArte(Artista) :-
    escribio(Artista, Obra),
    esComic(Obra).
```

La coma , es conjunción (Y lógico).

Tipos de variables en una regla:

- **Libre:** no está instanciada todavía al momento de la consulta. Ej: **Artista** cuando preguntamos quién escribió algo.
- **Ligada:** ya tiene un valor asignado por unificación. Ej: **Obra** queda ligada al iterar sobre la base de conocimiento.

```
esArtistaDeNovenoArte(Artista) :-
    escribio(Artista, Obra), % Obra queda ligada al unificarse con cada hecho
    esComic(Obra).
```

Inversibilidad

Capacidad de un predicado de funcionar con variables libres en cualquiera de sus argumentos.

```
% escribio es inversible: se puede preguntar por Artista u Obra
?- escribio(Artista, sandman).
?- escribio(gaiman, Obra).

% is NO es inversible: el lado derecho debe estar completamente ligado
siguiente(N, S) :- S is N + 1.
?- siguiente(41, S). % S = 42 → OK
?- siguiente(N, 42). % ERROR: N no está ligado
```

Desigualdad en Prolog

```
\= % desigualdad: se cumple si los dos términos NO unifican
hermano(Uno, Otro) :- padre(Alguien, Uno), padre(Alguien, Otro), Uno \= Otro.
```

Aritmética en Prolog

El = en Prolog **unifica**, no asigna ni compara numéricamente.

Para cálculos numéricos se usa **is**:

```
siguiente(N, S) :- S is N + 1.
?- siguiente(41, S). % S = 42
```

Backtracking: cuando Prolog no puede satisfacer una condición, vuelve al punto de elección anterior y prueba otra alternativa. Es el mecanismo de búsqueda de soluciones del motor de inferencia.

Cuantificador Universal

```
% forall(Condición, Propiedad): todos los que cumplen Condición, cumplen Propiedad
?- forall(habitat(Animal, Bioma), templado(Bioma)).
```

Diferencias Predicado Prolog vs Función Haskell

	Función Haskell	Predicado Prolog
Retorna	Un valor (unicidad)	Nada (solo V/F)
Respuestas	Exactamente una	Puede tener múltiples
Variables	Parámetros tipados	Incógnitas ligables
Inversible	No aplica	Sí (cuando es posible)

Relación Funtores con Orden Superior

Los **funtores** en Prolog encapsulan individuos compuestos (como `punto(3, 4)`), análogamente a cómo el orden superior en Haskell trata funciones como valores de primer orden. Ambos mecanismos permiten abstraer sobre estructura o comportamiento pasándolos como datos.

4. PREGUNTAS TIPO PARCIAL - RESPUESTAS DIRECTAS

Dos diferencias Lenguaje vs Paradigma:

1. El paradigma es una forma de pensar; el lenguaje es su implementación concreta.
2. Un lenguaje puede implementar múltiples paradigmas (lenguajes híbridos).

Lenguajes híbridos: Scala, Python, Kotlin, C++.

Función partida: función definida por condiciones (guardas) en lugar de por extensión.

```
f x | x > 0 = x | otherwise = -x
```

Dominio e imagen:

- `mealcanza largo = largo <= largoDeListon` → dominio: `Number`, imagen: `Bool`
- `cuantoMesobra cantidad = largoDeListon - cantidad` → dominio: `Number`, imagen: `Number`

Unicidad y Existencia: garantizan que las funciones se comporten como funciones matemáticas — siempre hay una respuesta (existencia) y es única por entrada (unicidad).

Principio de Sustitución: como una función siempre devuelve lo mismo ante los mismos argumentos, se puede reemplazar cualquier aplicación por su resultado (consecuencia de la transparencia referencial).

Pattern Matching: mecanismo que selecciona qué cláusula aplicar comparando los argumentos con patrones de arriba hacia abajo.

Inferencia de tipos: Haskell deduce automáticamente el tipo de una función analizando cómo se usan sus parámetros y qué operaciones se aplican sobre ellos.

Diferencia Tupla vs Data: la tupla agrupa valores sin nombre propio (pierde semántica); el Data define un tipo con nombre, constructor y funciones de acceso automáticas — es más expresivo.

Aplicación Parcial vs Composición: la aplicación parcial fija parámetros de una función generando otra con menos parámetros; la composición encadena funciones para que la salida de una sea la entrada de la siguiente.

Recursividad: función definida en términos de sí misma con un caso base que corta la recursión.

```
factorial 0 = 1
factorial n = n * factorial (n-1)
```

Orden Superior: funciones que reciben o retornan otras funciones. Ejemplos: `map`, `filter`, `foldr`, `any`, `all`.

Predicado vs Función Haskell: el predicado no devuelve valor, solo tiene valor de verdad y puede tener múltiples soluciones; la función siempre retorna exactamente un valor.

Evaluación EAGER: evalúa todos los argumentos antes de ejecutar la función (contrario a Lazy de Haskell).

Átomo, Hechos y Reglas:

- Átomo: individuo simple en Prolog (`juan`, `sandman`).
- Hecho: cláusula que expresa una verdad directa (`escribio(gaiman, sandman).`).
- Regla: cláusula que infiere una conclusión a partir de condiciones (`esComic(X) :- ...`).

Consultas individuales vs existenciales: la individual instancia todos los argumentos y devuelve V/F; la existencial deja variables libres y Prolog retorna los individuos que las satisfacen.

Aritmética y = en Prolog: = unifica (no compara numéricamente ni asigna). Para operar numéricamente se usa `is`. El `is` no es invertible: el lado derecho debe estar completamente ligado.

Funtores vs Orden Superior: ambos permiten tratar estructura o comportamiento como dato — en funcional son funciones de primer orden pasadas como argumento; en lógico son términos compuestos que encapsulan individuos dentro de un predicado.

Parcial Ej: Librería Haskell

```

{-
Resolver un problema para una librería
¿Es cierto que alguien escribió una determinada obra?
¿Quién o quienes escribieron una obra?
¿Qué obra escribió cierta persona?
Si es cierto que cierta persona escribió alguna obra, sin importar cual.
Si es cierto que cierta obra existe.

-}

import Data.List (find)

type Titulo = String
type Autor = String
data Obra = Obra { titulo:: Titulo, autores:: [Autor]} deriving (Show)

-- data base

obras :: [Obra]
obras = [ Obra "WheelOfTime" ["RobertJordan", "BrandonSanderson"],
          Obra "ASongOfIceAndFire" ["GeorgeRRMartin"] ]

hallarObraPorTitulo :: Titulo -> [Obra] -> Maybe Obra
hallarObraPorTitulo tituloObra = find (\obra -> titulo obra == tituloObra)

alguienEscribioLaObra :: Titulo -> [Obra] -> Bool
alguienEscribioLaObra tituloObra obras =
    maybe False (not . null . autores) (hallarObraPorTitulo tituloObra obras)

quienesEscribieronEstaObra :: Obra -> [Autor]
quienesEscribieronEstaObra = autores

quienesEscribieronUnaObra :: Titulo -> [Obra] -> [Autor]
quienesEscribieronUnaObra myTitulo todasObras = quienesEscribieronEstaObra (head
(filter (\ob -> titulo ob == myTitulo ) todasObras))

queObraEscribioEstaPersona :: Autor -> [Obra] -> Obra
queObraEscribioEstaPersona autor obras = head (filter (elem autor . autores)
obras)

estaPersonaEscribioUnaObra :: Autor -> [Obra] -> Bool
estaPersonaEscribioUnaObra autor = any (elem autor . autores )

existeEstaObra :: Titulo -> [Obra] -> Bool
existeEstaObra tituloObra = any (\ob -> tituloObra == titulo ob)

main :: IO()
main = do
    putStrLn ("Alguien escribio cierta obra" ++ show(alguienEscribioLaObra

```

```

"ASongOfIceAndFire" obras))
  putStrLn ("Quienes escribieron cierta obra" ++ show(quienesEscribieronUnaObra
"WheelOfTime" obras))
  putStrLn ("Que obra escribio cierta persona" ++ show(queObraEscribioEstaPersona
"RobertJordan" obras))
  putStrLn ("Cierta persona escribio una obra" ++ show(estaPersonaEscribioUnaObra
"RobertJordan" obras))
  putStrLn ("Existe cierta obra" ++ show (existeEstaObra "WheelOfTimea" obras))

```

Parcial Ej: Cartas Haskell

```

{-
Teniendo un juego de cartas con superhéroes y para cada superhéroe se tiene
información de sus poderes.

Se pide extender las cartas para incluir tags y definir funciones para cambiarlos

Dado un mazo de cartas:
Obtener los nombres de las cartas que comienzan con "bat"
Averiguar si hay cartas con los tags demasiado largos.
Corregir las cartas a las que pusieron tag #alguien en lugar de #alien
-}

data Carta = Carta {
  nombre :: String,
  velocidad :: Int,
  alutra :: Int,
  peso :: Int,
  fuerza :: Int,
  peleas :: Int,
  tags :: [String]
} deriving (Eq, Show)

cartas :: [Carta]
cartas = [ Carta "Superman" 9 8 8 10 132 ["#alien"],
           Carta "Batman" 7 7 8 8 210 ["#humano", "#alguien"],
           Carta "Flash" 10 9 7 9 31 ["#alguien"] ]

removeTag :: String -> Carta -> Carta
removeTag tag carta = carta { tags = filter (\t -> tag /= t) (tags carta) }

addTag :: String -> Carta -> Carta
addTag tag carta = carta { tags = tag : tags carta }

cartasQueEmpiezanConBat :: [Carta] -> [Carta]
cartasQueEmpiezanConBat = filter (\c -> take 3 (nombre c) == "bat")

cartasConTagsLargos :: [Carta] -> Int -> [Carta]
cartasConTagsLargos cartas limite = filter(any (\t -> length t > limite) . tags)
cartas

```

```

corregirCartasConTagAlguien :: [Carta] -> [Carta]
corregirCartasConTagAlguien = map(removeTag "#alguien" . addTag "#alien") . filter
(elem "#alguien" . tags)

main :: IO()
main = do
    putStrLn("Cartas que arrancan con 'bat': " ++ show(cartasQueEmpiezanConBat
cartas))
    putStrLn ""
    putStrLn("Cartas con tags largos: " ++ show(cartasConTagsLargos cartas 5))
    putStrLn ""
    putStrLn("Corregir cartas con tag #alguien: " ++
show(corregirCartasConTagAlguien cartas))
    putStrLn ""
    putStrLn("Cartas output: " ++ show cartas) -- haskell no modifica una
referencia, siempre genera una copia de los objetos o data types

```

Ejercicios Haskell

```

{-
Ejercicios en Haskell
Definir la función esMayorDeEdad (edad>=18)
Definir la función esMenorDeEdad (edad<18)
Definir NombreFormateado que toma un nombre y un apellido y devuelve el Apellido,
Nombre.
Verificar
EsMayorDeEdad 19
EsMayorDeEdad 17
:t EsMayorDeEdad
:t EsMayorDeEdad 17
Definir esMenorDeEdad partiendo de esMayorDeEdad

```

Ejercicios en Haskell

Queremos saber que velocidad alcanzará la pelota a los 5 segundos si se la deja caer desde una ventana ubicada en el último piso de un edificio.

Queremos saber que distancia recorre la pelota a los 5 segundos si se la deja caer desde una ventana ubicada en el último piso de un edificio.

Queremos saber si la pelota rebota contra el piso a los 5 segundos si se la deja caer desde una ventana a 80 m de altura.

```
-}
```

```

esMayorDeEdad :: Integer -> Bool
esMayorDeEdad x = x >= 18

```

```

esMenorDeEdad :: Integer -> Bool
esMenorDeEdad x = not (esMayorDeEdad x)

```

```

nombreFormateado :: String -> String -> String

```

```
nombreFormateado x y = y ++ ", " ++ x
```

```
gravity :: Float
gravity = 9.81
```

```
freeFallSpeed :: Float -> Float
freeFallSpeed x = x * gravity
```

```
freeFallDistance :: Float -> Float
freeFallDistance x = 1/2 * x*x * gravity
```

```
doesBounce :: Float -> Bool
doesBounce x = freeFallDistance x == 80
```

```
main :: IO ()
main = do
    print (esMayorDeEdad 19)
    print (esMayorDeEdad 17)

    print (esMenorDeEdad 19)
    print (esMenorDeEdad 17)
    print (nombreFormateado "Martin" "Schubert")

    putStrLn ("Velocidad en caida libre en 5 segundos: " ++ show (freeFallSpeed 5)
    ++ " m/s")

    putStrLn ("Distancia recorrida en 5 segundos " ++ show (freeFallDistance 5) ++ "
m")

    putStrLn ("Rebota la pelota a los 5 seg si se tira a 80m de altura? " ++ show
(doesBounce 5))
```

```
{-
```

```
— Ejercicios en Haskell —
```

```
- Elmayordelostres :: Ord a => a -> a -> a -> a
- Xor :: Bool -> Bool -> Bool
- Fibonacci :: Int -> Int
- Inferencia
    esMuchoMayor :: ???
    esMuchoMayor n m = n - m > 10
    funcionRara :: ???
    funcionRara n m = esMuchoMayor n (not m)
    f x y = g (h y (i x y) x) y
        f False " " = True
    funcionRara1 f = f 2 True
    funcionRara2 g = g.length
```

```

-}

elMayorDeLosTres :: Ord a => a -> a -> a -> a
elMayorDeLosTres x y z = max x (max y z)

xor :: Bool -> Bool -> Bool
xor a b
  | a && b = False
  | a || b = True
  | otherwise = False

{- — Otras formas de hacer un xor —

xor True False = True
xor False False = True
xor _ _ = False

xor a b = a /= b

xor a b = (a || b) && not (a && b)
-}

esMuchoMayor :: Int -> Int -> Bool
esMuchoMayor n m = n - m > 10

funcionRara :: Int -> Int -> Bool
funcionRara n m = esMuchoMayor n (negate m) -- no tiene sentido usar not como en
la consigna

-- f x y = g ( h y ( i x y ) x ) y
-- f False " " = True

-- f toma dos valores un numerico (float, int, etc) y un true y devuelve un tipo b
desconocido
f :: Num a => a -> Bool -> b
{- por ende funcionRara1 seria
ya que toma una funcion f que f toma un Num y un Bool
Input = Num a => (a -> Bool -> b)

y va a retornar lo que f retorne
Output = b

funcionRara1 es un wrapper de f que esta aplicando 2 y True a f
-}
funcionRara1 :: Num a => (a -> Bool -> b) -> b
funcionRara1 f = f 2 True

{-
funcionRara2 toma una funcion g y aplica
-}

```



```
funcionRara2 g = g.length

-- Abstracion, type classes y pattern matching
type Edad = Int
type Nombre = String
edad :: Nombre -> Edad
edad "Martin" = 21
edad _ = 0 -- cualquier otro matchea con 0

main :: IO ()
main = do

    print (elMayorDeLosTres 3 10 1)
    print (xor False True)
    putStrLn ("Es mucho mayor, debe ser true: " ++ show (esMuchoMayor 21 10))
    putStrLn ("Es mucho mayor, debe ser false: " ++ show (esMuchoMayor 19 10))

    putStrLn ("Funcion rara, debe ser true: " ++ show (funcionRara 19 10))
    putStrLn ("Funcion rara, debe ser false: " ++ show (funcionRara 5 1))

    putStrLn ("Edad de Martin: " ++ show (edad "Martin"))
```

```
fibonacci :: Int -> Int
```

```
fibonacci n
    | n > 1 = fibonacci (n - 1) + fibonacci (n - 2)
    | n == 0 = 0
    | otherwise = 1

{-
otro forma

fibonacci 0 = 0
fibonacci 1 = 1
fibonacci n = fibonacci (n - 1) + fibonacci (n - 2)

-}
```

```
main :: IO ()
main = do

    print (fibonacci 1)
    print (fibonacci 4)
```

```

data Estudiante = UnEstudiante {
    nombre :: String,
    legajo :: String,
    nota :: Int
} deriving (Eq, Show)

juan :: Estudiante
juan = UnEstudiante { legajo = "123", nombre = "juan", nota = 1}

juana :: Estudiante
juana = UnEstudiante { legajo = "123", nombre = "juan", nota = 2}

persona = [juan, juana]

main :: IO()
main = do

    putStrLn ( "Output: " ++ show (juan == juana))

```

Parcial Ej: Libreria prolog

```

/*
 * Resolver un problema para una librería
 * ¿Es cierto que alguien escribió una determinada obra?
 *     - escribio(stephenKing, it) -> true
 *     - escribio(stephenKing, socorro) -> false
 * ¿Quién o quienes escribieron una obra?
 *     - escribio(X, it) -> stephenKing
 *     - escribio(X, watchmen) -> alanMoore
 * ¿Qué obra escribió cierta persona?
 *     - escribio(elsaBornemann, Obra) -> socorro
 * Si es cierto que cierta persona escribió alguna obra, sin importar cual.
 *     - escribio(elsaBornemann, _) -> true
 * Si es cierto que cierta obra existe.
 *     - escribio(_, it) -> true
 */

escribio(elsaBornemann, socorro).
escribio(neilGaiman, sandman).
escribio(alanMoore, watchmen).
escribio(neilGaiman, americanGods).
escribio(neilGaiman, buensoPresagios).
escribio(terryPratchett, buensoPresagios).
escribio(brianAzarello, cienBalas).
escribio(warenElis, planetary).
escribio(frankMiller, elCaballeroOscuroRegresa).
escribio(frankMiller, batmanAnioUno).
escribio(isaacAsimov, fundacion).
escribio(isaacAsimov, yoRobot).
escribio(isaacAsimov, elFinDeLaEternidad).

```

```

escribio(isaacAsimov, laBusquedaDeLosElementos).
escribio(joseHernandez, martinFierro).
escribio(stephenKing, it).
escribio(stephenKing, misery).
escribio(stephenKing, carrie).
escribio(stephenKing, elJuegoDeGerald).
escribio(julioCortazar, rayuela).
escribio(jorgeLuisBorges, ficciones).
escribio(jorgeLuisBorges, elAleph).
escribio(horacioQuiroga, cuentosDeLaSelva).
escribio(horacioQuiroga, cuentosDeLocuraAmorYMuerte).

```

Ejercicios Prolog

```

/*
Ejercicio 1: Resolver un problema para una consultora
  ¿Es cierto que alguien programa en Cobol?
    programaEn(_, cobol).
  ¿Quién o quienes programan en Cobol?
    programaEn(X, cobol).
  ¿Qué lenguajes maneja María?
    programaEn(maria, X).
  ¿Maria y Mario son colegas?
    colegas(maria, mario).
  ¿Quienes son colegas de lenguajes?
    colegas(maria, mario).
*/

programaEn(maria, cobol).
programaEn(maria, java).
programaEn(mario, cobol).
programaEn(mario, python).
programaEn(jose, cobol).
programaEn(jorge, java).
programaEn(jorge, python).

colegas(X, Y) :- programaEn(X, Lang), programaEn(Y, Lang).

/*
Ejercicio 2: Se pide armar el árbol familiar Real Inglés, usando la relación
“esHijoDe”
  ¿Es cierto que la reina Isabel tiene hijos?
    esHijoDe(_, isabelIII).
  ¿Quién o quienes son hijos de la reina Isabel?
    esHijoDe(x, isabelIII).
  ¿Quiénes son los nietos del Carlos III?
    esHijoDe(x, carlosIII).
  ¿Es cierto que Archie es primo del príncipe Jorge?
    esPrimoDe(archie, jorge).
  ¿Quienes son primos?
    esPrimoDe(X, Y).
*/

```

```

*/

esHijoDe(carlosIII, isabelII).
esHijoDe(ana, isabelII).
esHijoDe(andrew, isabelII).
esHijoDe(edward, isabelII).
esHijoDe(henry, carlosIII).
esHijoDe(archie, henry).
esHijoDe(willy, carlosIII).
esHijoDe(jorge, willy).

esNietoDe(X, Y) :- esHijoDe(X, Z), esHijoDe(Z, Y).
esPrimoDe(X, Y) :- esNietoDe(X, Z), esNietoDe(Y, Z), esHijoDe(X, PadreX),
esHijoDe(Y, PadreY), PadreX \= PadreY.

/*
Ejercicio 3: Definir las reglas para resolver el siguiente problema:
En Haskell, la operación que está permitida sobre un tipo de dato depende de la
TypeClass a la que pertenece.
Cada type class tiene las siguientes operaciones permitidas.
La consulta es puedo usar tal operación en tal clase?

Los Num pueden ser sumados, restados, multiplicados.
Los Fractional pueden ser divididos
A los show se los puede mostrar

Relaciones:
    Los Int son Num y Show
    Los Float y Double son Fractional y Show
    Los Bool son solo Show
    Y todos los Fractional son Num

Utilizar operacionPermitida(Operación, ClaseTipo) y pertenece(Clase, ClaseTipo).
*/

operacionPermitida(num, sumados).
operacionPermitida(num, restados).
operacionPermitida(num, multiplicados).
operacionPermitida(fractional, divididos).

pertenece(int, num).
pertenece(num, show).

pertenece(float, fractional).
pertenece(double, fractional).
pertenece(fractional, num).
pertenece(fractional, show).

pertenece(bool, show).

% pertenece transitivo – hereda typeclasses de typeclasses
perteneceA(Tipo, Clase) :- pertenece(Tipo, Clase).
perteneceA(Tipo, Clase) :- pertenece(Tipo, Intermedio), perteneceA(Intermedio,

```

Clase).

puedeHacer(X, Y) :- perteneceA(X, Class), operacionPermitida(Class, Y).

/*

Ejercicio 4: Analizar la inversibilidad de los siguientes predicados:

%amigo(Uno, Otro)

amigo(nico, fernando).

amigo(axel, Persona) :- Amigo(Persona, nico).

amigo(alf, _).

%id(Algo, LoMismo).

id(X, X)

%mayorDeEdad(Persona)

mayorDeEdad(Persona) :- Edad > 18, edad(Persona, Edad).

TODOS SON INVERSIBLES.

*/

amigo(nico, fernando).

amigo(axel, Persona) :- amigo(Persona, nico).

amigo(alf, _).

id(X, X).

edad(pepe, 18).

mayorDeEdad(Persona) :- Edad > 18, edad(Persona, Edad).

/*

Ejercicio 5: Dado el predicado inversible padre/2 definir los predicados abuelo/2, hermano/2 y ancestro/2

El siguiente predicado hermano es inversible?

hermano(Uno, Otro) :- padre(Alguien, Uno), Uno \= Otro, padre(Alguien, Otro).

Como serían los predicados de Primo? Y de Tio?

*/

padre(hijo, padre).

hermano(Uno, Otro) :- padre(Alguien, Uno), Uno \= Otro, padre(Alguien, Otro).

primo(X, Y) :- padre(X, PadreX), padre(Y, PadreY), hermano(PadreX, PadreY).

tio(X, Y) :- padre(X, PadreX), hermano(PadreX, Y).

/*

Ejercicio 6: Dado los siguientes partidos responder:

Los Equipos africanos hacen muchos goles (goles>2).

Hubo una goleada (>3) entre dos Equipos?.

*/

partido(islandia, 5, croacia, 1).

partido(nigeria, 0, argentina, 1).

partido(croacia, 0, argentina, 6).

partido(brasil, 5, costaRica, 0).

partido(brasil, 2, croacia, 2).

```
partido(senegal,5,croacia,1).
continente(brasil,america).
continente(croacia,europa).
continente(argentina,america).
continente(costarica,america).
continente(islandia,europa).
continente(nigeria,africa).
continente(senegal,africa).
descalificado(brasil).
descalificado(alemania).

goleada(X, Y) :-
    partido(X, GolesX, Y, GolesY),
    GolesX - GolesY >= 3.

goleada(X, Y) :-
    partido(X, GolesX, Y, GolesY),
    GolesY - GolesX >= 3.

africanosConGoles(X) :-
    continente(X, africa),
    partido(X, Goles, _, _),
    Goles > 2.

africanosConGoles(X) :-
    continente(X, africa),
    partido(_, _, X, Goles),
    Goles > 2.
```